Taste Match SE — Architecture

Novelty Library v2.19 · admin reference · not served to readers

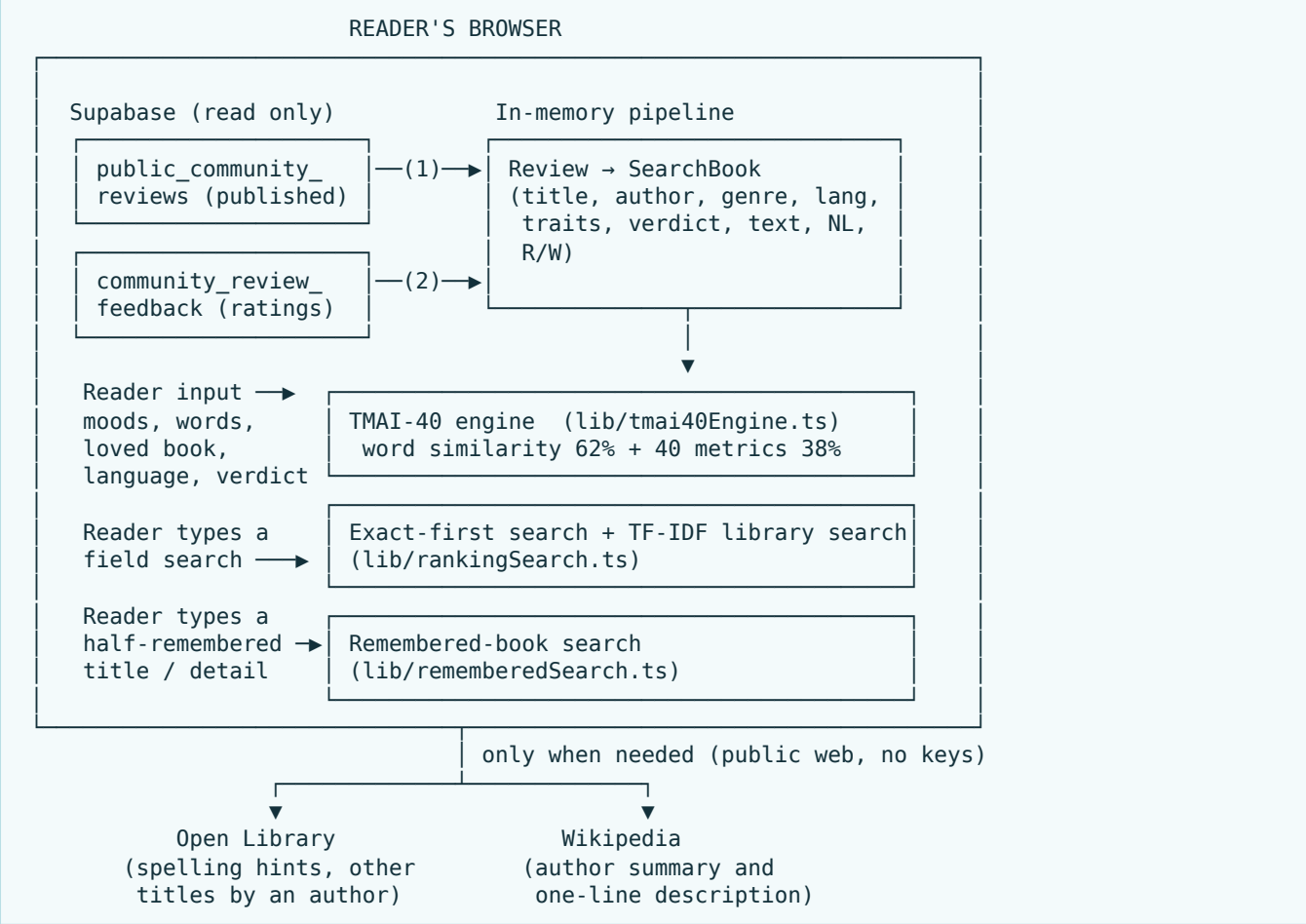
This document explains everything Taste Match does, which file does it, and which queries run. It is a description of the code as shipped in 2.19; nothing in this folder is imported by the web app, so it cannot affect the build or the live site.

---

1. One-paragraph summary

Taste Match SE suggests published reviews that fit what a reader feels like reading. All of the matching runs **in the reader's browser** over the published reviews the page has already loaded. It has no server of its own, never writes to Supabase, stores nothing about the reader's choices, and needs no account. Two optional lookups go to public websites (Open Library, Wikipedia) for spelling help and author notes.

2. Big picture



3. Where Taste Match appears

Place	What runs	Result size
Taste Match page `/taste-match`	TMAI-40 for moods, description, loved book, language, verdict; remembered-book search; match profile card	top 3 picks; 3 remembered-book hits
Live Rankings → free description box	TMAI-40 on the typed description	top 5
Live Rankings → field search	Exact-first search on one field (genre, language, traits, author) + wider TF-IDF matches	lists
Review page → "More like this"	TMAI-40 seeded only with that review as the loved book	top 3
Author page → "About the author"	Wikipedia + Open Library lookup	one note
Admin → Control Settings → Taste Match	explanation, backstage notes, 40-metric table, settings lab (sandbox)	admin only

4. Source map

File	Responsibility
`src/pages/TasteMatchPage.tsx`	The page. Loads published reviews and community ratings, computes the NL rating, shows the intro and the live counts, mounts the panel.
`src/components/RankingFinder.tsx`	`RankingSearch` (Live Rankings) and `TasteMatchPanel` (Taste Match page): all controls, the match-profile card, remembered-book box, results.
`src/lib/tmai40Engine.ts`	The ranking engine (`rankTmai40`) and the 40 metric definitions.
`src/lib/rankingSearch.ts`	Text normalisation, genre knowledge, exact-first field search, TF-IDF/cosine library search, moods, "did you mean", nearest books, the lab search used by admin.
`src/lib/rememberedSearch.ts`	Typo-tolerant search for half-remembered titles, authors and details.
`src/lib/authorBio.ts`	Open Library / Wikipedia lookups: author note and "did you mean" hints.
`src/components/AuthorBioBlock.tsx`	Draws the author note.
`src/components/SimilarReviews.tsx`	"More like this" on review pages.
`src/lib/readerTools.ts`	`usePublished()` and `reviewToSearchBook()`.
`src/lib/engagement.ts`	`computeNlRating()`.
`src/lib/reviews.ts`	`fetchReviews()` — the one Supabase read of published reviews.
`src/components/AdminTasteMatchPanel.tsx`	Admin explanation and settings lab.
`vercel.json`	Content-Security-Policy `connect-src` allows `openlibrary.org` and `en.wikipedia.org`.

5. Data in

5.1 Published reviews

`fetchReviews()` reads the privacy-safe view `public_community_reviews` (only `is_published = true`, newest first). The view omits reviewer emails and contact details. Drafts, pending and deleted reviews never reach the engine.

5.2 Reader ratings → NL rating

The Taste Match page also reads `community_review_feedback` (`review_id, rating`) for the loaded review ids. If that read fails, matching simply continues without community ratings.

NL rating = (sum of reader ratings + the reviewer's R/W rating) ÷ (number of reader ratings + 1). With no reader ratings it equals the R/W rating; with no R/W rating and no reader ratings it is empty.

5.3 What the engine sees of each review (`SearchBook`)

Title, author, genre, language, traits (cleaned), verdict, the first 4,000 characters of the review text, the NL rating and the R/W rating. The Taste Match page also passes translation, series and Goodreads/Amazon values when present. `metricValues` is reserved for future structured data and is empty today.

6. Normalising text

Everything is compared after: lower-casing, removing accents, reading "&" as "and", dropping punctuation and collapsing spaces. Common filler words (the, book, novel, story, read, like, want, looking, good, best…) are ignored when words are compared.

7. The TMAI-40 engine (`rankTmai40`)

7.1 Steps

1. **Build the wish.** The typed description, the words behind each selected mood (up to three) and the profile of the loved book (genre, traits, language, verdict) are joined into one text.
2. **Hard filters.** Language must match exactly if chosen. If verdicts are chosen the review's verdict must be one of them (reviews without a verdict are dropped). The loved book itself is never returned.
3. **Score 1 — word similarity.** Each review's text (title, author, genre, language, traits, verdict, translation, series and review text) becomes a word list. Word weights are TF-IDF: `(1 + ln count) × (ln((N+1)/(df+1)) + 1)`, length-normalised. The wish is weighted the same way and compared with every review by cosine similarity (0 to 1).
4. **Score 2 — the 40 metrics.** See 7.2. Only metrics that the wish (or loved book) actually asks about are scored.
5. **Combine.** With a wish and at least one scored metric: `0.62 × word similarity + 0.38 × metric similarity`. With a wish only: word similarity. With metrics only: metric similarity. With neither: `0.8 × (NL or R/W)/10 + 0.2 × R/W/10`.
6. **Relevance gate.** When there is a wish, anything below 0.015 combined is dropped, so a high rating can never turn an unrelated book into a match.
7. **Sort.** By score, then NL rating, then R/W rating, then title. The top *n* are returned. Score is shown as one decimal percentage; it is a relative ranking score, not a probability.

7.2 The 40 metrics

Each metric has a weight (0.2 to 1.5; genre is heaviest). Kinds:

- **Text metrics** (traits, genre, pace, writing style, atmosphere …) look for cue phrases in the review wording, plus the review's own field where one exists (genre, traits, language, verdict, translation). Match = F1 token overlap between what the wish asks for and what the review shows.
- **Numeric metrics** (emotional resonance, twist predictability, rereadability, poetic weight …) read *high* and *low* cue phrases in the review wording to estimate 0–1 as `(high + 0.5) / (high + low + 1)`. The wish sets a target (0.82 if it uses a high cue, 0.18 for a low cue, 0.5 for both). Score = `1 − |gap|`.
- **Metadata-only metrics** (book popularity bias, regional frequency mapping, PQI, per-capita sales, author popularity bias, narrative draft probe score) have no honest signal in review text. They are skipped until structured catalog data exists.
- A loved book contributes its own metric profile: **70% from the typed wish, 30% from the loved book** when both exist.
- A metric with no evidence in a review is skipped, not counted as a miss. Only a direct field that exists and does not match (for example a different genre) counts as a real zero.

The complete list with weights is in the admin panel table and in `src/lib/tmai40Engine.ts` (`TMAI40_METRICS`).

7.3 Reasons shown with each pick

"Echoes your words" (word similarity ≥ 4%), "Shares the story feel" (any metric ≥ 25%), "Language: …", "Verdict: …"; fallbacks "Loved by readers" or "Closest fit on the shelf". Maximum four.

7.4 Surprise me

Picks randomly from the top 12 matches; with no wish it picks from Perfection / Go for it reviews (in the chosen language), then from the whole shelf if that is empty.

8. Moods

Nine fixed moods, each a bundle of concept groups and words: Mind-bending, Edge-of-seat, Emotional, Romantic, Epic & adventurous, Dark & intense, Light & fun, Thought-provoking, Inspiring. Up to three can be active; their words join the wish text.

9. Library search (`rankingSearch.ts`)

9.1 Exact-first field search (`strictSearch`)

One chosen field (genre, language, traits, author) and a short query. Scores: exact word 100, known equivalent 92 (e.g. "psychological suspense thriller" = "psychological thriller"), phrase 85, contained 80, all words present 75, typo-tolerant 60. For genre and traits, 22 hidden genre groups provide the equivalents.

9.2 Wider matches (`broadSearch`, TF-IDF + cosine)

Free text over title, author, genre, traits, language, verdict and review text. Field weights: genre 3, traits 3, title 2, author 2, language 2, verdict 1, review text 1 (first 600 terms). Concept tags add weight 3 for the review's own genre group and 0.8 for neighbouring groups (the 30 loose pairs such as psychological ↔ suspense ↔ crime). Term frequency is sublinear, inverse document frequency is smoothed, vectors are length-normalised. Minimum similarity 5% (8% for "variants" of a direct search).

9.3 Combined (`runSearch`)

Direct hits first (ordered by the description as tiebreaker), then wider matches that exclude them.

9.4 Fall-backs

- **Did you mean**: edit-distance and word-similarity against the library's real values for that field.
- **Nearest books**: wide search at ≥ 0.5%; if still nothing, reader favourites so the box is never a dead end.
- Typo allowance: 1 typo for words of 5–8 letters, 2 for 9+.

10. Remembered-book search (`rememberedSearch.ts`)

Purpose: "harry poter", "frank herbart", "that book with the octopus".

- Filler words are ignored; every remaining word is compared with the title words (weight 3), author words (weight 2.5) and the genre/traits/review words (weight 1.6, multiplied by how rare the word is across the library).
- Word fit: exact 1, starts-with 0.85 (3+ letters), typo 0.7 (1 typo for 4–7 letters, 2 for 8+).
- Whole-title closeness (edit distance) rescues near-spellings; title containing the whole query scores 0.95.
- A result needs at least half of the words to hit (or a close whole-title match) and at least 22%.
- Up to three results, each with a reason ("Title looks like…", "Author looks like…", "Mentions…").

11. Match-profile card (Taste Match page)

Appears when there is a wish and picks. It is a **summary only**: a headline from the chosen moods (or "Readers of "), the most common first genres and traits among the top three picks, and chips for loved book, language and verdicts. It never feeds back into ranking.

12. Public-web lookups (the only network calls beyond Supabase)

Trigger	Request	Used for
Remembered-book box, no local result, ≥ 4 characters, after 600 ms	Open Library `search.json?q=<text>&limit=3`	Real title/author hints; if a hint spells like one of our reviews it is shown, otherwise named as not reviewed yet
Author page opens	Wikipedia `w/api.php` (search + extracts + description)	One-line description, 4-sentence summary (trimmed to about 420 characters)
Author page opens	Open Library `search.json?author=<name>&sort=editions`	Other titles and publication years

Rules: the Wikipedia page is accepted only if its title contains the author's name and its text reads like a writer; otherwise nothing is shown. Results are cached in memory for the visit only. Requests have a 6-second timeout and fail silently. These requests go **from the reader's browser**, so Wikipedia / Open Library see the reader's IP address; nothing is sent to Novelty Library's own servers. Wikipedia text is attributed with a link.

13. Admin tools

The admin tab shows the explanation, a backstage dump, the 40-metric table (with how many have evidence in today's reviews) and a **settings lab** that runs the real TF-IDF engine with changed weights next to the live settings. The lab is a sandbox: it changes nothing readers see.

14. What is stored, what is sent

Item	Where
Moods, typed words, loved book, language, verdicts, clicks	Nowhere. In-page state only; gone on refresh.
Published reviews and community ratings	Read from Supabase (read only).
Typed text for spelling help / author name	Sent to Open Library / Wikipedia only in the cases in section 12.
Writes to Supabase by Taste Match	None.

15. Performance and safety notes

- Cost grows with the number of published reviews (the engine scores every review per search); typing is debounced with React's deferred values.
- Scores are display-only; they never change a review's published ratings.
- The folder `taste-match/` holds documentation only and is outside `src/`, so it is not compiled or deployed.

16. Limits worth knowing

- Story-feel metrics are read from review wording, so a review that never mentions pace or atmosphere gives those metrics no evidence.
- Matching reads published reviews only; a book with no review cannot be recommended.
- Open Library and Wikipedia can be slow or unavailable; the page works without them.

17. Change log

2.2 TMAI-40 engine · 2.11 icon and polish · 2.15 "What is Taste Match SE?" intro with live counts · 2.16 remembered-book search and match-profile card · 2.17 Open Library hints and author note · 2.18 surname-first author names, admin backstage dump · 2.19 this folder (architecture + query catalog).